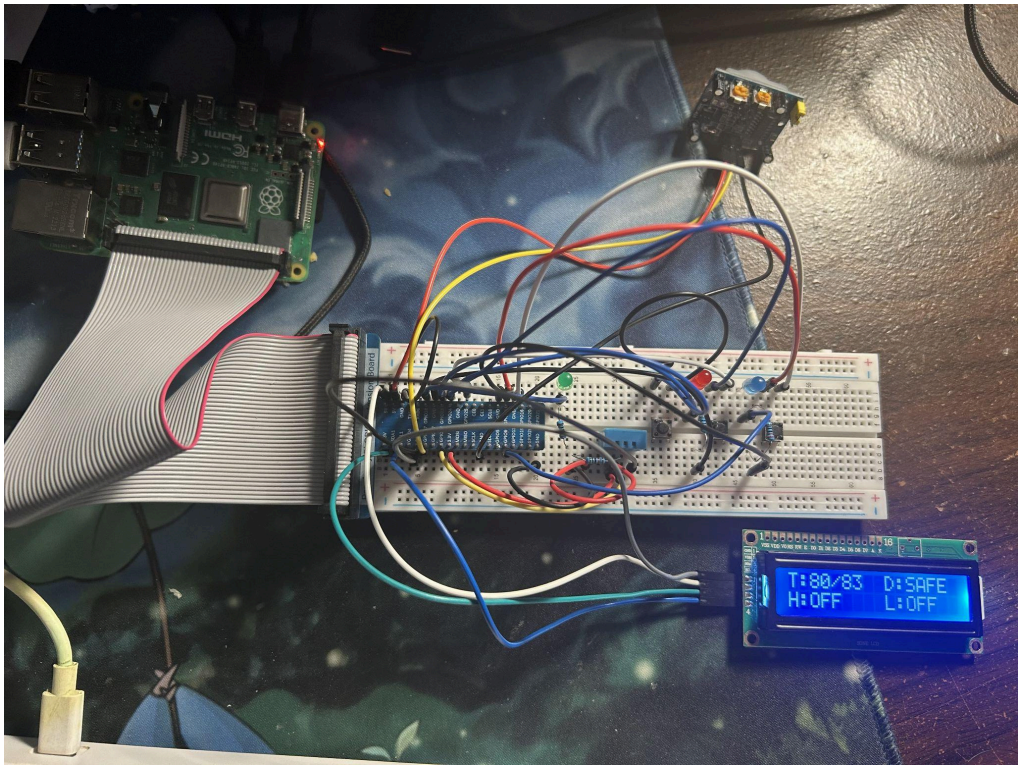
EECS 113 Final Project
Kenny Le
88293067



Introduction

This project consists of a Building Management System that simulates the scenario of a computer-based control system installed in buildings that controls and monitors the building's mechanical and electrical equipment such as ventilation, lighting, power systems, fire systems, and security systems by utilizing a Raspberry Pi. In order to achieve these functionalities, components such as DHT11 and PIR sensors, along with the LCD, will be the main components that will support measurements of temperature, humidity, and movement. Additionally, the use of buttons and LEDs will help visualize and support functions of adjusting desired temperature and simulating cases where a door or window is opened.

Materials

• Temperature and humidity sensor (DHT-11)
• PIR sensor
• LCD
• Push buttons
• LEDs
• Jump wires
• 10kΩ / 220kΩ resistors

Setup

<u>DHT-11 Sensor</u>: measuring temperature and humidity to compare with desired temperature
• assigned to pin 15 (GPIO 22)
<u>PIR Sensor</u>: tracking movement to turn on light for 10 seconds before turning off again
• assigned to pin 11 (GPIO 17)
<u>LCD Sensor</u>: displaying/monitoring data received by DHT, PIR, and CIMIS
• assignments noted via schematic
<u>CIMIS Data</u>: data regarding weather received via CIMIS to compare with actual temperature
• API Key + url connection

Schematics of Main Components

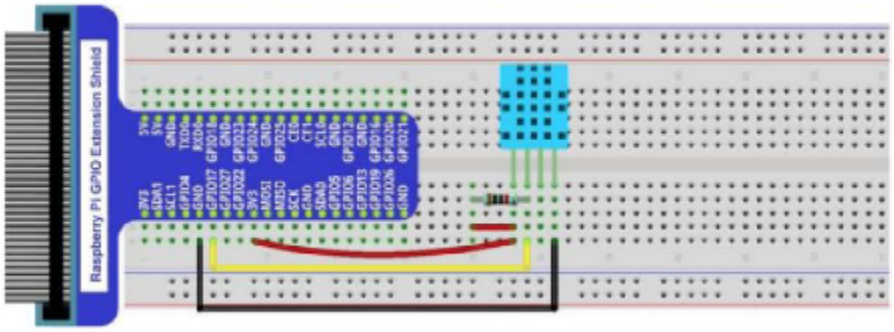


Figure 1: DHT-11 w/ 10kΩ resistor [figure schematic: GPIO 17 -> actual schematic: GPIO 22]

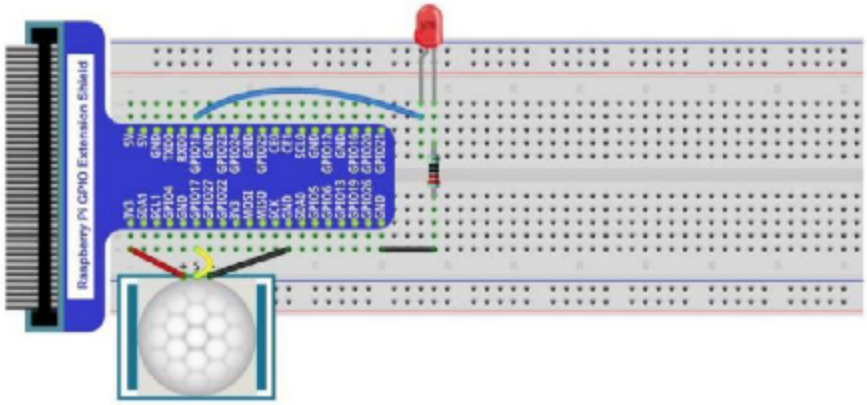


Figure 2: PIR Sensor [GPIO 17]

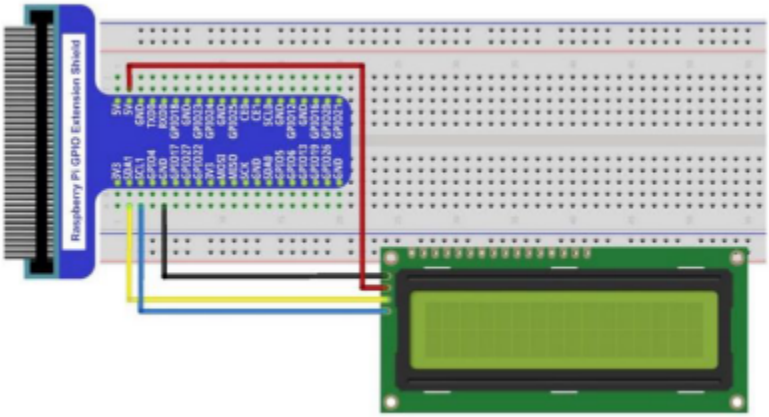


Figure 3: I2C LCD

Source Code

<u>Provided through Freenove</u>
- Adafruit_LCD1602.py
- DHT11.py
- Freenove_DHT.py
- PCF8574.py

<u>Function Codes</u>
- ambient_light.py
- hvac.py
- lcd_manager.py
- main.py

ambient_light.py

• responsible for managing movement detection and displaying these via LEDs

```
def turn_on_light():
    global timer, light_status
    if light_status:  # Light is already on, no need to update
        return
    if timer is not None:
        timer.cancel()
    logging.info('Turning on light')
    set_main_menu_message('ON', 'light')
    write_temporary_message("Light is on", duration=3)
    print("Motion detected!")
    GPIO.output(ledPin, GPIO.HIGH)
    light_status = True  # Update the status
    timer = threading.Timer(10, turn_off_light)
    timer.start()

def turn_off_light():
    global timer, light_status
    if not light_status:  # Light is already off, no need to update
        return
    GPIO.output(ledPin, GPIO.LOW)
    print("No motion detected")
    set_main_menu_message('OFF', 'light')
    write_temporary_message("Light is off", duration=3)
    logging.info('Turning off light')
    light_status = False  # Update the status
    timer = None
```

Figure 4: Light function for handling motion

The purpose of turn_on_light() and turn_off_light() is to provide necessary behaviors when light is on or off. For example, light is on when motion is detected so therefore it will create a temporary message that light is on and also turn on the LED to indicate that it's on. When no motion is detected after 10 seconds, it switches to the turn off function and turns off light.

hvac.py

• responsible for the HVAC system functionality, measuring temperature to compare with desired temperature, and utilizes CIMIS data

```
def get_humidity_from_CIMIS(app_key=api_key, targets='75', start_date='2024-06-08', end_date='2024-06-08')
    # TODO: implement function to retrieve humidity data from CIMIS system
    base_url = 'https://et.water.ca.gov/api/data?appKey=daaee995-eb8e-4ac3-80c7-3d229b153ad2&targets=75&st
    params = {
        'appKey': app_key,
        'targets': targets,
        'startDate': start_date,
        'endDate': end_date,
    }

    response = requests.get(base_url)
```

Figure 5: CIMIS data handling using API Key, Targeting, Dates, and URL

The purpose of get_humidity_from_CIMIS() is to format a url using API key, targeting, and dates in order to receive weather information from a specific location. For example, “targets=’75’” refers to the weather station for Irvine, so therefore information regarding weather will be about Irvine.

```
def read_temperture() -> float:
    for i in range(0, 5):
        chk = dht.readDHT11()
        if (chk is dht.DHTLIB_OK):
            return dht.temperature
```

Figure 6: Temperature reading function using DHT11

The purpose of read_temperature() is to read temperatures and humidity being received from the DHT11 sensor. “dht.DHTLIB_OK” is utilized to confirm that the temperature was sensed by the DHT11 with no issues.

```
def door_action(channel):
    global CURRENT_DOOR_STATE
    if CURRENT_DOOR_STATE == DOOR_STATE.CLOSED:
        CURRENT_DOOR_STATE = DOOR_STATE.OPEN
        set_main_menu_message('OPEN', 'door')
        write_temporary_message("Door is open", duration=3)
        print('Door is open')
        logging.info('Door is open')
        time.sleep(2)
        turn_off_hvac()
        return
    else:
        CURRENT_DOOR_STATE = DOOR_STATE.CLOSED
        set_main_menu_message('SAFE', 'door')
        write_temporary_message("Door is closed", duration=3)
        print('Door is closed')
        logging.info('Door is closed')
```

Figure 7: Door function for HVAC system

The purpose of door_action() is to provide the functionality of doors that open or close when being called to do so. For example, if the door button is pressed, the state of the door or window will change from one state to another.

```python
def adjust_desired_temp(channel):
    global desired_temp, weather_index
    print('Button is pressed')
    if channel == UP_BUTTON_PIN:
        desired_temp = min(95, desired_temp + 1)
        print('desired temp is now ' + str(desired_temp))
        message = str(desired_temp) + '/' + str(weather_index)
        set_main_menu_message(message, 'temp')
        write_temporary_message("Adjusted Temp:" + str(desired_temp), duration=3)
    else:
        desired_temp = max(65, desired_temp - 1)
        print('desired temp is now ' + str(desired_temp))
        message = str(desired_temp) + '/' + str(weather_index)
        set_main_menu_message(message, 'temp')
        write_temporary_message("Adjusted Temp:" + str(desired_temp), duration=3)
```

Figure 8: Desired temperature function used to compare with actual temperature

The purpose of adjust_desired_temp() is to implement output when buttons that are designed to lower or heighten temperature are pressed. If the down button is pressed, the desired temperature will drop by one. On the other hand if the up button is pressed, the opposite will occur where the desired temperature will go up by one.

```python
def calculate_weather_index(temperature, humidity) -> int:
    temperature = temperature * 9 / 5 + 32
    return round(temperature + 0.05 * humidity)
```

Figure 9: Weather index via calculating temperature by using humidity from CIMIS data

The purpose of calculate_weather_index() is to calculate temperature using an algorithm to get an accurate temperature. It's important to note that humidity is received through CIMIS data to be compared to.

```python
def turn_on_AC():
    global hvac_start_time
    hvac_start_time = time.time()
    GPIO.output(BLUE_LED_PIN, GPIO.HIGH)
    GPIO.output(RED_LED_PIN, GPIO.LOW)
    logging.info("AC is on")
    set_main_menu_message('AC', 'hvac')
    write_temporary_message("AC is on", duration=3)
    time.sleep(1.5)

def turn_on_heater():
    global hvac_start_time
    hvac_start_time = time.time()
    GPIO.output(RED_LED_PIN, GPIO.HIGH)
    GPIO.output(BLUE_LED_PIN, GPIO.LOW)
    logging.info("Heater is on")
    set_main_menu_message('HEAT', 'hvac')
    write_temporary_message("Heater is on", duration=3)
    time.sleep(1.5)

def turn_off_hvac():
    global hvac_start_time
    hvac_start_time = time.time()
    GPIO.output(RED_LED_PIN, GPIO.LOW)
    GPIO.output(BLUE_LED_PIN, GPIO.LOW)
    logging.info("HVAC is off")
    set_main_menu_message('OFF', 'hvac')
    write_temporary_message("HVAC is off", duration=3)
    time.sleep(1.5)
```

Figure 10: AC, Heater, and HVAC function for HVAC system

The purpose of turn_on_AC(), turn_on_heater(), and turn_off_hvac() is to provide the necessary functionality when being called for in certain situations within the HVAC system. The function, "turn_on_AC()", replicates the need for A/C if the area is hot so then it will only turn on if the actual temperature is above 3 degrees the desired temperature. This is oppositely the same for turn_on_heater as well where it needs to be 3 degrees below the desired temperature to need a heater. Turn_off_hvac() is called when the door or window is open so therefore HVAC does not need to be up.

```python
def control_hvac(temperature)-> Type[HVAC_STATE]:
    global current_weather_index, weather_index, desired_temp, CURRENT_DOOR_STATE, CURRENT_HVAC_STATE
    # print current hvac state
    print(f"Current HVAC state: {CURRENT_HVAC_STATE}")
    # format date for CIMIS API query parameters
    now = datetime.now() - timedelta(days=2)
    formatted_date = now.strftime('%Y-%m-%d')
    # get humidity from CIMIS, loop until return value is not None since CIMIS API is unreliable
    humidity = None
    while humidity is None:
        humidity = get_humidity_from_CIMIS(targets='75', start_date=formatted_date, end_date=formatted_date
    print(f"Current humidity: {humidity}")
    weather_index = calculate_weather_index(temperature, humidity)
    if current_weather_index != weather_index:
        current_weather_index = weather_index
        set_main_menu_message(str(desired_temp) + '/' + str(weather_index), 'temp')
        write_temporary_message("New WI:" + str(weather_index), duration=1.5)
    print(f"Current weather index: {weather_index}")
    if 90 < weather_index: # fire alarm system is on
        write_message("Fire! HVAC OFF" + '\n' + 'Door/Window OPEN')
        logging.info('Fire alarm is started')
        CURRENT_DOOR_STATE = DOOR_STATE.OPEN
        while(calculate_weather_index(round(read_temperture()), humidity) > 90):
            flash_leds()
        logging.info('Fire alarm is stopped')
        set_main_menu_message('', '')
        set_main_menu_message('SAFE', 'door')
        set_main_menu_message('COOL', 'hvac')
        write_temporary_message("Door is closed", duration=3)
        time.sleep(2)
        CURRENT_DOOR_STATE = DOOR_STATE.CLOSED
        turn_on_AC()
        return HVAC_STATE.COOL
    elif CURRENT_DOOR_STATE == DOOR_STATE.OPEN:
        # turn off HVAC
        return HVAC_STATE.OFF
    elif weather_index > desired_temp + 3:
        # turn on AC
        if CURRENT_HVAC_STATE != HVAC_STATE.COOL:
            turn_on_AC()
        return HVAC_STATE.COOL
    elif weather_index < desired_temp - 3:
        # turn on heater
        if CURRENT_HVAC_STATE != HVAC_STATE.HEAT:
            turn_on_heater()
        return HVAC_STATE.HEAT
    else:
        # turn off HVAC
        if CURRENT_HVAC_STATE != HVAC_STATE.OFF:
            turn_off_hvac()
        return HVAC_STATE.OFF

def flash_leds():
# flash both LEDs for one second periodically
    GPIO.output(RED_LED_PIN, GPIO.HIGH)
    GPIO.output(BLUE_LED_PIN, GPIO.HIGH)
    GPIO.output(GREEN_LED_PIN, GPIO.HIGH)
    time.sleep(1)
    GPIO.output(RED_LED_PIN, GPIO.LOW)
    GPIO.output(BLUE_LED_PIN, GPIO.LOW)
    GPIO.output(GREEN_LED_PIN, GPIO.LOW)
    time.sleep(1)
```

Figure 11: HVAC states and weather conditions (Fire alarm implementation)

The purpose of control_hvac() is to calculate actual humidity and display a fire alarm scenario if the area seems to be too hot. It's vital to note that the temperature that triggers the fire alarm is set at 90 degrees. Alongside this, flash_leds() is used to flash when a fire alarm system is occurring.

Functions

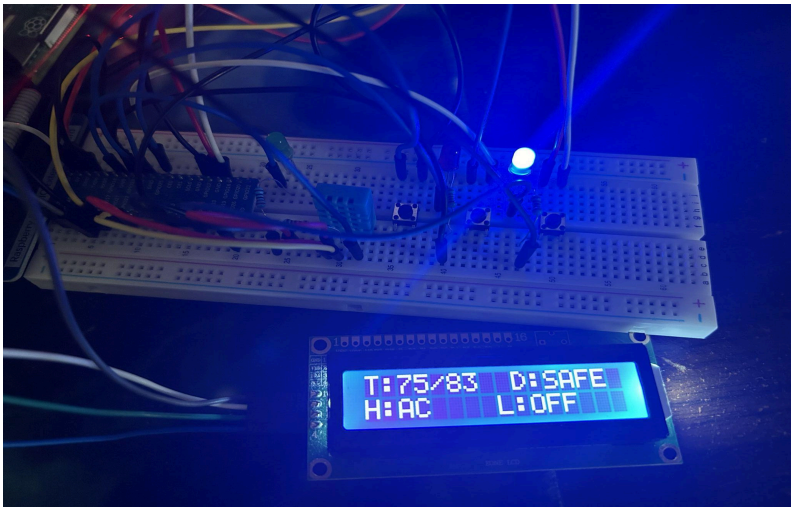


Figure 12: A/C is on (75 desired, 83 actual) and Door is safe/closed

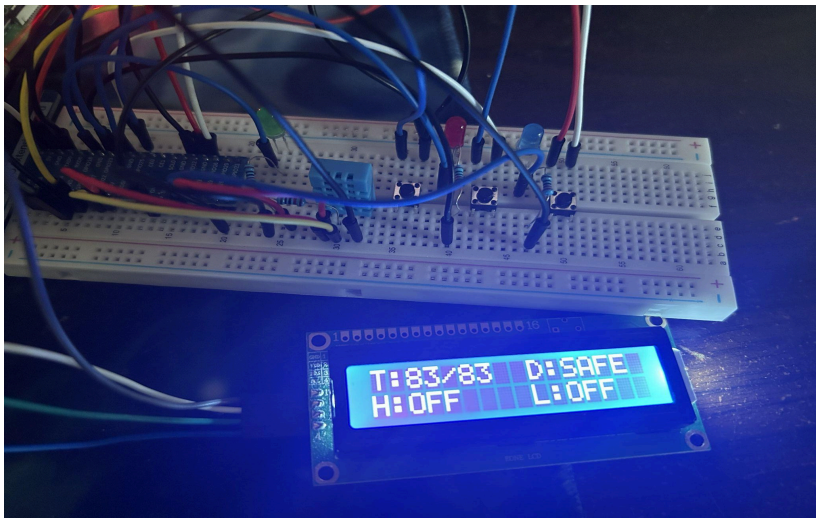


Figure 13: No A/C or Heater (83 desired, 83 actual) and Door is safe/closed

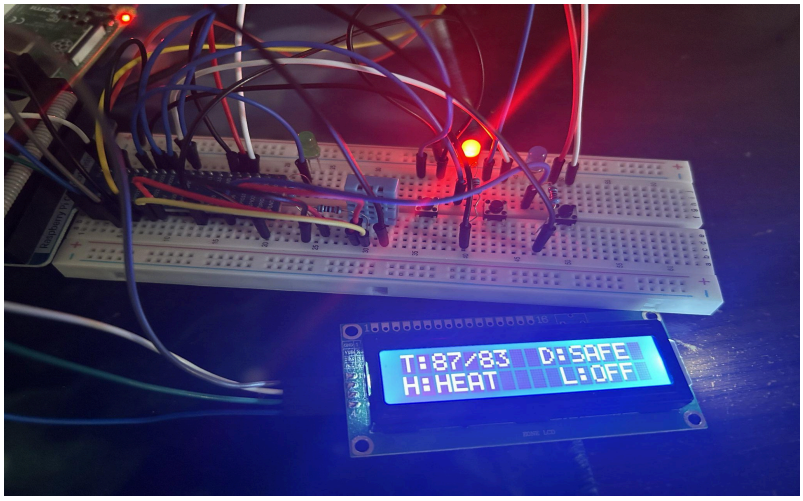


Figure 14: Heater is on (87 desired, 83 actual) and Door is safe/closed

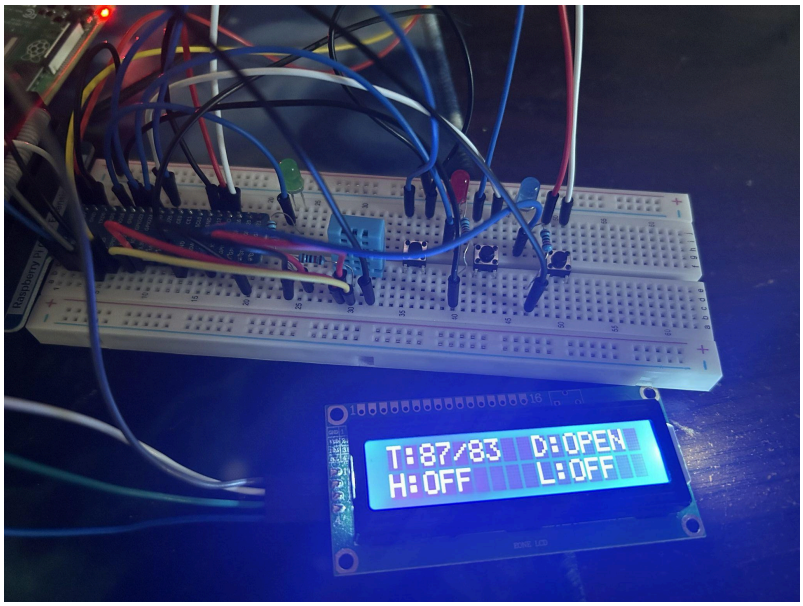


Figure 15: No A/C or Heater (87 desired, 83 actual) and Door is open

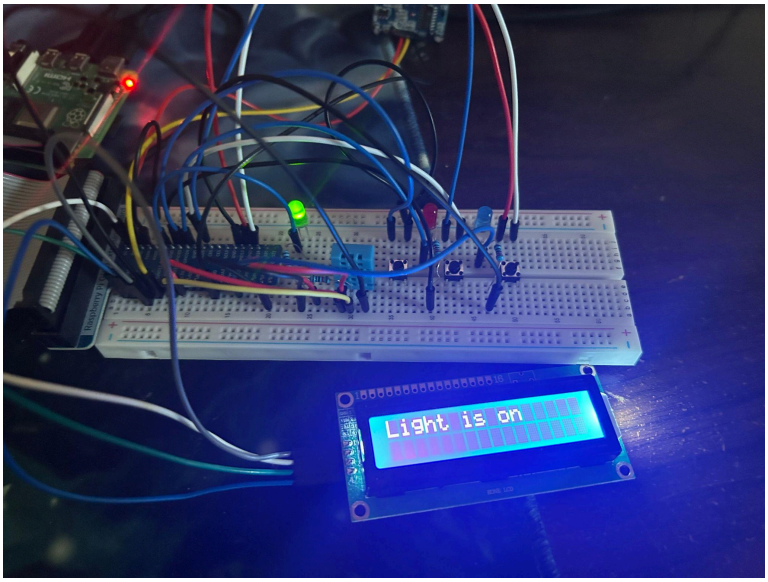


Figure 16: Light is on with PIR Sensor when movement was shown

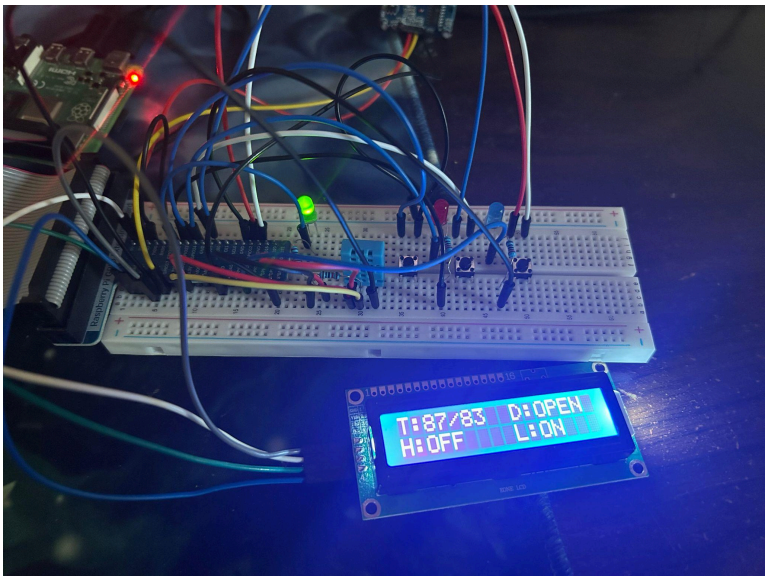


Figure 17: Light is on with PIR Sensor when movement was shown (2)

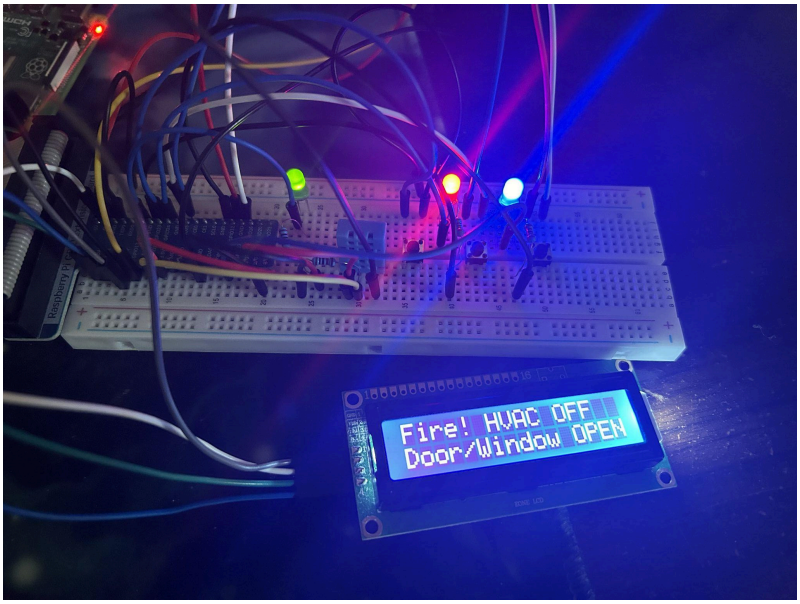


Figure 18: Fire alarm system (designed temperature: 75, actual temperature = 91)